

API операционных систем

Введение, файловый ввод-вывод

Подготовил
Старший преподаватель
МИЭМ НИУ ВШЭ
Морозов Владимир Игоревич

Основные понятия

- API операционной системы – главным образом, набор системных вызовов, позволяющих пользователю запрашивать у ОС необходимые действия
- Системный вызов (упрощённо) – некоторая функция, выполняемая операционной системой в режиме ядра
- Системные вызовы необходимы, т.к. пользователю в целях безопасности не предоставлен прямой доступ к важным компонентам ОС, с которыми ему нужно работать (например, к устройствам ввода-вывода, средствам управления ресурсами системы и т.д.)

Основные отличия от классического программирования

Отличия от классического программирования

- API ОС представлен в основном набором функций в стиле Си, оперирующих примитивными типами данных, указателями или структурами. ООП не используется.
- API как правило использует собственные типы данных (однако эти типы определены через базовые типы Си с помощью **#define** или **typedef**)
- Ввиду отсутствия ООП, доступ к объектам, как правило, осуществляется через числовые идентификаторы
- Для установки тех или иных параметров работы функций используются битовые флаги (будут рассмотрены отдельно)

Linux

Обработка ошибок

Способы обработки ошибок в Linux

- Возвращаемым значением большинства системных вызовов является число
- Существует соглашение, по которому число, большее или равное нулю, свидетельствует об успешном завершении вызова, а число меньше нуля (как правило, `-1`) говорит об ошибке
- Однако в случае ошибки возвращаемое значение системного вызова не содержит информации о том, какая именно ошибка возникла. Чтобы получить такую информацию, необходимо проверить значение `errno`, в котором хранится код текущей ошибки.

Пример обработки ошибок в Linux

- В представленном листинге кода показан пример обработки ошибки, возникающей при попытке открытия несуществующего файла с помощью системного вызова `open`

```
int file = open("hello", O_RDONLY);  
if (file < 0)  
{  
    printf("Error %d\n", errno);  
    return 1;  
}
```

- Чтобы работать с `errno`, нужно подключить заголовочный файл `errno.h`

Способы обработки ошибок в Linux

- Для более удобной обработки ошибок Linux предоставляет функции, работающие с их текстовыми описаниями:
 - **perror** – принимает на вход строку-подсказку и выводит в стандартный поток ошибок строку формата:
строка_подсказка: описание_последней_ошибки
 - **strerror** – принимает в качестве аргумента номер ошибки и возвращает Си-строку, содержащую её (ошибки) текстовое описание
- Чтобы воспользоваться функцией **strerror**, нужно подключить заголовочный файл `string.h`

Пример обработки ошибок с текстовым описанием

- В представленном листинге кода показан пример обработки ошибки, возникающей при попытке открытия несуществующего файла с помощью системного вызова `open`

```
int file = open("hello", O_RDONLY);  
if (file < 0)  
{  
    perror("Error occurred");  
    char * errMsg = strerror(errno);  
    printf("Error occurred: %s\n", errMsg);  
    return 1;  
}
```

Результат обработки ошибок с текстовым описанием

- Результатом работы отрывка кода, приведённого на предыдущем слайде будет строка следующего вида, выведенная дважды:
Error occurred: *описание_ошибки*

Константы с номерами ошибок

- В системной библиотеке имеется набор констант, каждая из которых содержит значение, равное номеру одной из ошибок.
- В Linux такие константы имеют названия, начинающиеся с буквы **E**, к примеру, ошибка, которую мы получили в предыдущем примере, имеет код, содержащийся в константе **ENOENT**
- Эти константы можно использовать для проверки того, какая именно ошибка стала причиной невыполнения системного вызова и изменять поведение программы в зависимости от этого
- Список констант и их значений можно найти в документации

Заключительные замечания по обработке ошибок

- Обработку ошибок необходимо производить **всегда!** (отсутствие обработки ошибок в ходе выполнения практических работ будет считаться ошибкой)
- `errno` содержит код только **последней** возникшей ошибки. Коды предыдущих ошибок утрачиваются.
- Некоторые системные вызовы могут возвращать значение `-1` в качестве легального (не ошибочного). Это стоит предусматривать, обнуляя перед таким вызовом значение `errno`.

Файловый ввод-вывод

Основные положения

- Доступ к файлу из программы осуществляется по его **дескриптору** – числовому идентификатору, однозначно связанному с файлом
- Для осуществления базовых операций файлового ввода-вывода в Linux используются 4 системных вызова:
 - **open** – открывает файл, имя которого передано в качестве аргумента и возвращает дескриптор открытого файла
 - **read** – читает необходимое количество байтов из файла в буфер (массив байтов)
 - **write** – записывает необходимое количество байтов из буфера в файл
 - **close** – закрывает файл
- Существует больше системных вызовов для работы с файлами, в рамках данного занятия рассматриваются только основные
- Для использования данных системных вызовов нужно подключить заголовочный файл `fcntl.h`

Системный вызов open

- **open** принимает два или три аргумента:
 - Имя файла (или путь к файлу), который нужно открыть, в виде строки в стиле Си
 - Флаги, задающие параметры работы с файлом (будут рассмотрены далее)
 - Права доступа к новому файлу (если данный системный вызов используется для создания нового файла)
- **open** возвращает дескриптор открытого файла в случае успешного выполнения и **-1** в случае ошибки
- При использовании данного системного вызова гарантируется, что возвращённый дескриптор будет наименьшим возможным дескриптором среди используемых в данной программе

Битовые флаги

Основные понятия

- Для указания параметров работы функции (например, системного вызова) используются битовые флаги – целые числа, в которых все биты установлены в 0 и только один в 1, к примеру, так мог бы выглядеть в двоичной записи битовый флаг: 00000100
- Позиция единичного бита в флаге играет ключевую роль – именно в ней закодировано, какую опцию добавляет данный флаг
- Как правило флаги для каждой из функций представлены в виде набора констант, имена которых позволяют судить о назначении каждого из флагов
- При передаче флагов функции они все объединяются в одно число с помощью операции логического ИЛИ

Флаги системного вызова open

- Системный вызов `open` имеет множество флагов, определяющих специфику его работы. Здесь перечислены только основные:
 - `O_RDONLY` – открыть файл только для чтения
 - `O_WRONLY` – открыть файл только для записи
 - `O_RDWR` – открыть файл для чтения и записи
 - `O_CREAT` – если файл с указанным именем не существует, создать его
 - `O_APPEND` – вести запись в конец файла (если данный флаг не установлен, запись ведётся в начало; то, что было записано ранее, перезаписывается)
 - `O_TRUNC` – удалить всё, что есть в файле, перед записью
 - `O_EXCL` – (используется совместно с `O_CREAT`) обязательно **создать новый** файл; если файл с таким именем уже существует, выдать ошибку

Флаги системного вызова open

- Нельзя комбинировать с помощью логического ИЛИ флаги `O_RDONLY` и `O_WRONLY`, т.к. они не соответствуют общепринятому соглашению о флагах. Поэтому для выполнения и чтения и записи используется отдельный флаг.

Флаги `O_WRONLY` и `O_TRUNC`

Особое внимание следует уделить флагу `O_WRONLY`, а именно порядку, в котором производится запись в существующий файл в случае, если он открыт с этим флагом.

Если в файле уже имелась информация, и он будет открыт с таким флагом, то новая информация будет помещена в начало файла. При этом, если размер новых данных меньше тех, что уже были в файле, часть старых данных также останется.

Например, если открыть файл, содержащий строку “Hello” с флагом `O_WRONLY` и записать в него строку “abc”, в результате в файле окажется строка “abclo”. Чтобы избежать этого поведения, необходимо пользоваться флагом `O_TRUNC`.

Права доступа к файлу

- Если системный вызов `open` в результате создаёт новый файл, третьим аргументом **необходимо** задать права доступа к нему
- Права доступа представляются 9-битным числом, как принято в Linux:
 - Первая тройка отвечает за права на чтение, запись и выполнение для создателя (владельца) файла
 - Вторая тройка – за те же права для участников группы владельца файла
 - Третья – за те же права для всех остальных пользователей
- Предопределённые битовые значения хранятся в константах, начинающихся с `S_I` (например, `S_IRWXU` или `S_IXUSR`)
- Для задания всех необходимых прав константы объединяются как битовые флаги
- Если при создании файла данное значение не будет задано, права доступа к файлу будут заданы случайным образом

Пример использования флагов в системном вызове open

- В данном примере системный вызов open откроет файл file.txt для записи в конец. Если такой файл не существовал, он будет создан.

```
int file = open("file.txt", O_WRONLY | O_APPEND |  
O_CREAT, S_IRGRP | S_IRWXU);
```

Системный вызов read

- **read** принимает три аргумента:
 - Дескриптор открытого для чтения файла
 - Указатель на буфер для считанной информации (массив байт), **память должна быть выделена заранее!**
 - Количество байт, которые необходимо считать
- **read** возвращает количество считанных байт информации в случае успешного вызова и **-1** в случае ошибки
- Для работы с системным вызовом **read** нужно подключить заголовочный файл `unistd.h`
- Запрос количества байтов большего, чем размер выделенной под буфер памяти, приводит к неопределённому поведению и **должен избегаться**

Пример использования read

- В данном примере из файла file.txt будет считана строка символов размером 5 байт:

```
int file = open("file.txt", O_RDONLY);
if (file < 0) { perror("Error while opening file");
return 1;}
char * rdbuf = malloc(5); // Выделено 5 байт динамической
памяти
ssize_t bytesRead = read(file, rdbuf, 5);
if (bytesRead < 0) { perror("Error while reading file");
return 1;}
printf("Read string: %s\n", rdbuf);
free(rdbuf);
```

Системный вызов write

- **write** принимает три аргумента:
 - Дескриптор открытого для записи файла
 - Указатель на буфер с записываемой информацией (массив байт)
 - Количество байт, которые необходимо записать
- **write** возвращает количество записанных байт информации в случае успешного вызова и **-1** в случае ошибки
- Для работы с системным вызовом **write** нужно подключить заголовочный файл `unistd.h`
- Запись количества байтов большего, чем размер выделенной под буфер памяти, приводит к неопределённому поведению и **должна избегаться**

Пример использования write

- В данном примере в файл file.txt будет записана строка символов размером 5 байт:

```
int file = open("file.txt", O_WRONLY);
if (file < 0) { perror("Error while opening file");
return 1;}
char * wrbuf = "adcd"; // Пятый байт – символ конца
строки
ssize_t bytesWrite = write(file, wrbuf, 5);
if (bytesWrite < 0) { perror("Error while writing file");
return 1;}
```

Консольный ввод-вывод

- В Linux существует три предопределённых файловых дескриптора, которые уже открыты к моменту запуска программы:
 - `STDIN_FILENO` – для стандартного потока ввода
 - `STDOUT_FILENO` – для стандартного потока вывода
 - `STDERR_FILENO` – для стандартного потока ошибок
- В соответствии с правилом о номерах дескрипторов они имеют значения 0, 1 и 2 соответственно
- Таким образом, чтобы осуществить консольный ввод или вывод, следует передать эти дескрипторы вызовам `read` или `write`
- Сообщения об ошибках следует выводить в поток ошибок, т.к. различные среды исполнения в таком случае получают возможность обрабатывать этот вывод особым образом (например, выводить в отдельный файл)

Системный вызов close

- `close` принимает один аргумент – дескриптор открытого файла
- Несмотря на то, что в малых учебных программах, целью которых является только тренировка определённых навыков программирования, закрытие файла кажется лишённым смысла, необходимо всегда это делать, чтобы при разработке более масштабных программ не оставлять файлы открытыми
- Закрывать файлы в процессе работы программы нужно отчасти потому, что изменения, внесённые в файл, применяются только после закрытия, а также по ряду других важных причин

Документация

- Получить подробнейшее руководство по тому или иному системному вызову можно, используя следующую команду в терминале:
man 2 название_системного_вызова
- Также можно воспользоваться поисковыми системами

Полезная литература

- Майкл Керриск – Linux API. Исчерпывающее руководство. – Книга, на основании которой сделана данная презентация. Для закрепления материала рекомендуются главы 3 и 4. Для более углублённого ознакомления – главы 5 и 13.